

DAY 6

PyTorch Fundamentals

A complete practical and mathematical guide to tensors, autograd, neural networks, training loops, DataLoaders, evaluation, devices, saving/loading, and the PyTorch mental model.

Sprint	ML-21_DAY_SPRINT
Focus	Deep Learning / PyTorch
Level	Foundations
Goal	Move from neural networks from scratch to framework-based implementation

Core principle: PyTorch should not feel like magic. Every important API in this guide is connected back to the mathematics and backpropagation you implemented manually in the previous days.

Table of Contents

1. What PyTorch is and why it matters
2. Installation and imports
3. Tensors: the foundation
4. Tensor shapes, dimensions, dtypes, indexing, and reshaping
5. Tensor mathematics and matrix multiplication
6. Autograd and computational graphs
7. Parameters and the manual neural-network connection
8. nn.Module and building neural networks
9. nn.Linear and activations
10. Loss functions
11. Optimizers
12. The PyTorch training loop
13. Mini-batches, Dataset, and DataLoader
14. Train / validation / test workflow
15. Training vs evaluation mode and torch.no_grad
16. CPU vs GPU and devices
17. Model evaluation and classification metrics
18. Saving and loading models
19. Complete end-to-end binary classifier
20. Mapping Days 2-5 concepts to PyTorch
21. Common mistakes and debugging checklist
22. Day 6 exercises and completion checklist

1. What PyTorch Is and Why It Matters

PyTorch is a deep learning framework centered around tensor computation, automatic differentiation, neural-network modules, optimizers, and hardware acceleration. The key idea for this sprint is that PyTorch automates repetitive parts of neural-network implementation while preserving the mathematical structure you have already learned.

On Days 2-5, you explicitly implemented matrix multiplication, activations, loss functions, derivatives, backpropagation, and parameter updates. PyTorch provides reliable abstractions for those same operations.

Concept you learned	PyTorch abstraction	What it represents
Vectors / matrices	<code>torch.Tensor</code>	Numerical data and parameters
$XW + b$	<code>nn.Linear</code>	Learnable affine transformation
Activation	<code>torch.relu</code> / <code>torch.sigmoid</code>	Nonlinear transformation
Loss	<code>nn.BCEWithLogitsLoss</code> / <code>nn.MSELoss</code> / <code>nn.CrossEntropyLoss</code>	Objective to minimize
Chain rule	<code>loss.backward()</code>	Automatic gradient computation
Gradient descent	<code>optimizer.step()</code>	Parameter update
Clear gradients	<code>optimizer.zero_grad()</code>	Prevent gradient accumulation
GPU computation	<code>.to(device)</code>	Move tensors/models to accelerator

Important: learning PyTorch is not replacing the mathematics. It is learning a tool that implements the mathematics efficiently and safely.

2. Installation and Imports

In Google Colab, PyTorch is usually already installed. Verify the installation:

```
import torch

print(torch.__version__)
```

The two most common namespaces for basic neural-network work are:

```
import torch
import torch.nn as nn
import torch.optim as optim
```

The alias `nn` is conventionally used for `torch.nn`, while `optim` is conventionally used for `torch.optim`.

3. Tensors: The Foundation

A tensor is PyTorch's primary numerical data structure. You can think of it as a generalized NumPy array with support for automatic differentiation and device acceleration.

3.1 Scalar

```
x = torch.tensor(5)

print(x)
print(x.ndim)
print(x.shape)
```

A scalar has zero dimensions. Its shape is empty because it does not contain rows or columns.

3.2 Vector

```
x = torch.tensor([1, 2, 3])

print(x)
print(x.ndim)
print(x.shape)
```

The shape is `torch.Size([3])`, meaning one dimension containing three elements.

3.3 Matrix

```
x = torch.tensor([
    [1, 2],
    [3, 4],
    [5, 6]
])

print(x)
print(x.ndim)
print(x.shape)
```

The shape is `torch.Size([3, 2])`: three rows and two columns.

3.4 Higher-dimensional tensors

```
x = torch.tensor([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
])

print(x.shape)
```

This has shape `[2, 2, 2]`. Higher-dimensional tensors are essential for images, sequences, batches, and other structured data.

4. Tensor Shapes, Dimensions, dtypes, Indexing, and Reshaping

4.1 Shape and ndim

```
x = torch.randn(3, 4, 5)

print(x.shape) # torch.Size([3, 4, 5])
print(x.ndim)  # 3
```

When debugging neural networks, shape inspection is one of your most important tools.

4.2 Data types

```
x = torch.tensor([1, 2, 3])

print(x.dtype)

x = torch.tensor(
    [1, 2, 3],
    dtype=torch.float32
)

print(x.dtype)
```

Neural-network calculations commonly use `float32` tensors. Integer tensors are useful for labels, indexing, and other tasks, but most trainable numerical computation uses floating-point values.

4.3 Creating useful tensors

```
zeros = torch.zeros(3, 4)
ones = torch.ones(2, 3)
random_uniform = torch.rand(3, 4)
random_normal = torch.randn(3, 4)

print(zeros.shape)
print(ones.shape)
print(random_uniform.shape)
print(random_normal.shape)
```

4.4 Indexing and slicing

```
X = torch.tensor([
    [10., 20., 30.],
    [40., 50., 60.],
    [70., 80., 90.]
])

print(X[0])      # first row
print(X[:, 0])   # first column
print(X[1, 2])   # one element
print(X[:2, :2]) # top-left 2x2 block
```

4.5 Reshaping

```
x = torch.tensor([
    [1, 2],
    [3, 4],
    [5, 6]
])

print(x.shape)

x = x.reshape(2, 3)

print(x)
print(x.shape)
```

Reshaping changes the view of the same underlying values when possible. The total number of elements must remain unchanged: $3 \times 2 = 2 \times 3 = 6$.

5. Tensor Mathematics and Matrix Multiplication

5.1 Element-wise operations

```
a = torch.tensor([1., 2., 3.])
b = torch.tensor([4., 5., 6.])

print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

The `*` operator above performs element-wise multiplication. This is different from matrix multiplication.

5.2 Matrix multiplication

```
A = torch.tensor([
    [1., 2.],
    [3., 4.]
])

B = torch.tensor([
    [5., 6.],
```

```

    [7., 8.]
])

C = A @ B

print(C)

```

The @ operator performs matrix multiplication. `torch.matmul(A, B)` is another equivalent form.

5.3 Neural-network shape reasoning

```

X = torch.randn(100, 5)
W = torch.randn(5, 3)

Z = X @ W

print(X.shape)
print(W.shape)
print(Z.shape)

```

The dimensions follow the rule $(100 \times 5)(5 \times 3) = (100 \times 3)$. This is exactly the linear algebra behind a dense layer.

6. Autograd and Computational Graphs

Autograd is PyTorch's automatic differentiation system. It records operations involving tensors that require gradients and uses that computation history to calculate derivatives.

6.1 Basic derivative

```

x = torch.tensor(
    3.0,
    requires_grad=True
)

y = x ** 2

y.backward()

print(x.grad)

```

Mathematically, $y = x^2$ gives $dy/dx = 2x$. At $x = 3$, the gradient is 6.

6.2 What `requires_grad` means

`requires_grad=True` tells PyTorch to track operations on the tensor because gradients with respect to that tensor may be required later.

6.3 A slightly richer example

```

x = torch.tensor(2.0, requires_grad=True)

y = 3 * x**2 + 2 * x + 1

y.backward()

print("y =", y.item())
print("dy/dx =", x.grad.item())

```

The derivative is $6x + 2$. At $x = 2$, it is 14. PyTorch calculates this automatically.

6.4 Gradients are stored

```

print(x.grad)

```

For a leaf tensor such as a trainable parameter, the gradient is commonly accessible through `.grad` after `backward()`.

7. Parameters and the Manual Neural-Network Connection

Before using `nn.Module`, it is useful to see that PyTorch can reproduce the neural network you implemented manually.

```
import torch
import torch.nn as nn

X = torch.tensor([
    [1., 2.],
    [3., 4.],
    [5., 6.]
])

y = torch.tensor([
    [0.],
    [1.],
    [1.]
])

W = torch.randn(2, 1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)

z = X @ W + b
predictions = torch.sigmoid(z)

loss_fn = nn.BCELoss()
loss = loss_fn(predictions, y)

loss.backward()

print("dL/dW:")
print(W.grad)

print("dL/db:")
print(b.grad)
```

This is the direct bridge from your previous work: forward pass -> loss -> backward pass -> gradients. The remaining step is to use an optimizer to update `W` and `b`.

8. nn.Module and Building Neural Networks

`nn.Module` is the base class used to define neural-network models in PyTorch. You define trainable layers in `__init__` and define data flow in `forward()`.

```
class NeuralNetwork(nn.Module):

    def __init__(self):
        super().__init__()

        self.layer1 = nn.Linear(2, 4)
        self.layer2 = nn.Linear(4, 1)

    def forward(self, x):

        x = self.layer1(x)
        x = torch.relu(x)
        x = self.layer2(x)

        return x
```

Instantiate it with:

```
model = NeuralNetwork()

print(model)
```

The model now contains two learnable linear layers and one ReLU activation.

8.1 Understanding `__init__`

The `__init__` method defines the components of the model. `super().__init__()` initializes the `nn.Module` machinery so PyTorch can discover parameters and submodules.

8.2 Understanding forward

`forward()` defines the computation performed when you call `model(X)`. You normally do not call `forward()` directly; use `model(X)`, which invokes the module correctly.

```
X = torch.randn(8, 2)

output = model(X)

print(output.shape)
```

Because the final layer has one output neuron, the output shape is `[8, 1]` for a batch of eight samples.

9. nn.Linear and Activations

9.1 nn.Linear

`nn.Linear(in_features, out_features)` represents a learnable affine transformation. Conceptually it performs a form of $XW^T + b$ using its internally stored weight and bias.

```
layer = nn.Linear(2, 4)

print(layer.weight.shape)
print(layer.bias.shape)
```

For `nn.Linear(2, 4)`, PyTorch stores the weight with shape `[4, 2]` and bias with shape `[4]`.

9.2 Inspecting parameters

```
for name, param in model.named_parameters():
    print(name, param.shape)
```

9.3 Counting parameters

```
total_params = sum(
    p.numel()
    for p in model.parameters()
)

print(total_params)
```

9.4 ReLU

```
x = torch.tensor([-2., -1., 0., 1., 2.])

y = torch.relu(x)

print(y)
```

ReLU is $\max(0, x)$. It introduces nonlinearity so a network can model more than a single linear transformation.

10. Loss Functions

The loss function measures how different the model's predictions are from the target values. The choice depends on the task.

Task	Common loss	Typical model output
Binary classification	<code>nn.BCEWithLogitsLoss()</code>	One raw logit per sample
Multiclass classification	<code>nn.CrossEntropyLoss()</code>	One logit per class
Regression	<code>nn.MSELoss()</code>	Continuous numerical output

10.1 Binary classification

```
loss_fn = nn.BCEWithLogitsLoss()
```

```
logits = model(X)
loss = loss_fn(logits, y)
```

`BCEWithLogitsLoss` combines sigmoid and binary cross-entropy in a numerically stable implementation. Therefore, when using this loss, the model should normally return raw logits rather than applying sigmoid inside the model.

10.2 Regression

```
loss_fn = nn.MSELoss()
```

```
predictions = model(X)
loss = loss_fn(predictions, y)
```

10.3 Multiclass classification

```
loss_fn = nn.CrossEntropyLoss()
```

```
logits = model(X)
loss = loss_fn(logits, class_labels)
```

For `CrossEntropyLoss`, `class_labels` are normally integer class indices rather than one-hot vectors.

11. Optimizers

An optimizer updates the model's parameters using the gradients calculated during backpropagation.

11.1 SGD

```
optimizer = optim.SGD(
    model.parameters(),
    lr=0.01
)
```

Conceptually, SGD applies $\theta \leftarrow \theta - \text{learning_rate} * \text{gradient}$.

11.2 Adam

```
optimizer = optim.Adam(
    model.parameters(),
    lr=0.001
)
```

Adam is an adaptive optimizer widely used in deep learning. For learning the mechanics, SGD is especially useful because its update rule maps directly to the gradient-descent equation.

12. The PyTorch Training Loop

The training loop is the central pattern you should be able to explain from memory.

```
for epoch in range(1000):

    # 1. Forward pass
    predictions = model(X)

    # 2. Calculate loss
    loss = loss_fn(predictions, y)

    # 3. Clear old gradients
    optimizer.zero_grad()

    # 4. Backpropagation
    loss.backward()

    # 5. Update parameters
    optimizer.step()
```

The five steps are: forward -> loss -> zero gradients -> backward -> update.

12.1 Why zero_grad?

PyTorch accumulates gradients by default. Without clearing them, the gradient from a new batch would be added to gradients left over from earlier batches.

12.2 Why backward?

loss.backward() traverses the recorded computation graph and calculates derivatives of the loss with respect to tensors that require gradients, including the model's trainable parameters.

12.3 Why optimizer.step?

optimizer.step() uses the gradients to change the model's parameters according to the optimizer's update rule.

12.4 Monitoring training

```
for epoch in range(1000):

    logits = model(X)
    loss = loss_fn(logits, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 100 == 0:
        print(
            f"Epoch {epoch}, "
            f"Loss: {loss.item():.4f}"
        )
```

loss.item() extracts a Python number from a scalar tensor for logging.

13. Mini-Batches, Dataset, and DataLoader

Real datasets are rarely trained by feeding every example through the model at once. Instead, training usually uses mini-batches. PyTorch provides Dataset and DataLoader utilities for this workflow.

13.1 TensorDataset

```
from torch.utils.data import TensorDataset

dataset = TensorDataset(X, y)
```

```
print(len(dataset))
print(dataset[0])
```

TensorDataset pairs corresponding tensors so that each index returns one input-target pair.

13.2 DataLoader

```
from torch.utils.data import DataLoader

loader = DataLoader(
    dataset,
    batch_size=32,
    shuffle=True
)
```

batch_size controls how many examples are processed at a time. shuffle=True randomizes training order each epoch.

13.3 Mini-batch training loop

```
for X_batch, y_batch in loader:

    predictions = model(X_batch)

    loss = loss_fn(predictions, y_batch)

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()
```

This is the same training logic as before, but now the model processes a small batch at a time.

13.4 Why mini-batches?

- They reduce the memory required for each forward/backward pass.
- They make training practical for large datasets.
- They introduce useful stochasticity into gradient-based optimization.
- They allow the same training pattern to scale to real-world datasets.

14. Train / Validation / Test Workflow

A complete machine-learning experiment should separate data according to purpose.

Split	Purpose
Training	Used to learn model parameters.
Validation	Used during development to compare configurations and tune decisions.
Test	Used for final, held-out evaluation after development decisions are made.

A common conceptual split is 70/15/15 or 80/10/10, but the correct proportions depend on dataset size and the problem.

14.1 Random split with PyTorch

```
from torch.utils.data import random_split

train_size = int(0.8 * len(dataset))
test_size = len(dataset) - train_size
```

```

train_dataset, test_dataset = random_split(
    dataset,
    [train_size, test_size]
)

train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=32,
    shuffle=False
)

```

For a more rigorous workflow, create a validation split as well and make sure preprocessing decisions do not leak information from validation/test data into training.

15. Training vs Evaluation Mode and torch.no_grad

15.1 model.train()

```
model.train()
```

This puts the model in training mode. It matters for layers such as Dropout and BatchNorm.

15.2 model.eval()

```
model.eval()
```

This puts the model in evaluation mode. It changes the behavior of layers that behave differently during inference.

15.3 torch.no_grad()

```

model.eval()

with torch.no_grad():
    logits = model(X)
    probabilities = torch.sigmoid(logits)

```

During inference, gradients are usually unnecessary. `torch.no_grad()` prevents PyTorch from tracking those operations for gradient computation, reducing memory use and computation.

15.4 Standard inference pattern

```

model.eval()

with torch.no_grad():
    logits = model(X)
    probabilities = torch.sigmoid(logits)
    predictions = (probabilities >= 0.5).float()

```

16. CPU vs GPU and Devices

PyTorch tensors and models can run on CPUs or compatible accelerators such as CUDA GPUs.

```

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print(device)

```

16.1 Move the model

```
model = model.to(device)
```

16.2 Move the data

```
x = x.to(device)
y = y.to(device)
```

16.3 Training on the selected device

```
model = model.to(device)

for X_batch, y_batch in train_loader:

    X_batch = X_batch.to(device)
    y_batch = y_batch.to(device)

    logits = model(X_batch)
    loss = loss_fn(logits, y_batch)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

The model and the tensors involved in the same operation must be on compatible devices. A common error is to leave the model on GPU while the data remains on CPU, or vice versa.

17. Model Evaluation and Classification Metrics

Loss is useful for optimization, but classification projects should usually report task-level metrics as well.

17.1 Accuracy

Accuracy is the fraction of predictions that are correct. It is easy to interpret but can be misleading for heavily imbalanced datasets.

```
correct = (predictions == y).sum().item()
accuracy = correct / len(y)

print(accuracy)
```

17.2 Precision, recall, and F1

Precision answers: among predicted positives, how many were actually positive? Recall answers: among actual positives, how many did the model identify? F1 is the harmonic mean of precision and recall.

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)

accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
cm = confusion_matrix(y_true, y_pred)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1:", f1)
```

```
print("Confusion matrix:")
print(cm)
```

For Day 6, focus on understanding what each metric means and how it is calculated from predictions. A deeper treatment of imbalanced classification and metric selection can come later.

18. Saving and Loading Models

Once a model is trained, you need a way to persist its learned parameters.

18.1 Save state_dict

```
torch.save(
    model.state_dict(),
    "model.pth"
)
```

`state_dict()` contains the model's learnable parameters and relevant persistent buffers.

18.2 Load parameters

```
model = NeuralNetwork()

model.load_state_dict(
    torch.load("model.pth")
)

model.eval()
```

The architecture must be recreated before loading its state dictionary.

18.3 Recommended mental model

```
# Save learned parameters
torch.save(model.state_dict(), "model.pth")

# Later:
model = NeuralNetwork()
model.load_state_dict(torch.load("model.pth"))
model.eval()
```

19. Complete End-to-End Binary Classifier

The following project puts the Day 6 concepts together. The dataset is a tiny AND-style binary classification problem so the mechanics remain visible.

19.1 Imports and data

```
import torch
import torch.nn as nn
import torch.optim as optim

from torch.utils.data import (
    TensorDataset,
    DataLoader
)

X = torch.tensor([
    [0., 0.],
    [0., 1.],
    [1., 0.],
    [1., 1.]
])
```

```

y = torch.tensor([
    [0.],
    [0.],
    [0.],
    [1.]
])

```

19.2 Dataset and DataLoader

```

dataset = TensorDataset(X, y)

loader = DataLoader(
    dataset,
    batch_size=4,
    shuffle=True
)

```

19.3 Model

```

class NeuralNetwork(nn.Module):

    def __init__(self):
        super().__init__()

        self.layer1 = nn.Linear(2, 4)
        self.layer2 = nn.Linear(4, 1)

    def forward(self, x):

        x = self.layer1(x)
        x = torch.relu(x)
        x = self.layer2(x)

        return x

model = NeuralNetwork()

```

19.4 Loss and optimizer

```

loss_fn = nn.BCEWithLogitsLoss()

optimizer = optim.SGD(
    model.parameters(),
    lr=0.1
)

```

19.5 Training

```

epochs = 1000

for epoch in range(epochs):

    model.train()

    for X_batch, y_batch in loader:

        logits = model(X_batch)

        loss = loss_fn(
            logits,
            y_batch
        )

        optimizer.zero_grad()

        loss.backward()

        optimizer.step()

```

```

if epoch % 100 == 0:
    print(
        f"Epoch {epoch}, "
        f"Loss: {loss.item():.4f}"
    )

```

19.6 Evaluation

```

model.eval()

with torch.no_grad():

    logits = model(X)

    probabilities = torch.sigmoid(logits)

    predictions = (
        probabilities >= 0.5
    ).float()

print("Probabilities:")
print(probabilities)

print("Predictions:")
print(predictions)

```

19.7 What happened internally?

For every mini-batch, the model computed $XW + b$ through its Linear layers, applied ReLU, produced a logit, computed binary cross-entropy with logits, built a computational graph during the forward pass, calculated gradients during `backward()`, and updated all trainable parameters during `optimizer.step()`.

This is the same learning algorithm you implemented manually earlier, but PyTorch now manages the parameter registration, gradient bookkeeping, differentiation, and optimization machinery.

20. Mapping Days 2-5 Concepts to PyTorch

Previous concept	Mathematical form	PyTorch form
Linear transformation	$z = XW + b$	<code>nn.Linear(...)</code>
Sigmoid	$1 / (1 + e^{-z})$	<code>torch.sigmoid(z)</code>
ReLU	$\max(0, z)$	<code>torch.relu(z)</code>
Loss	Task-specific function	<code>nn.*Loss</code> classes
Derivative	$dL/d\theta$	<code>parameter.grad</code> after <code>backward()</code>
Backpropagation	Chain rule	<code>loss.backward()</code>
Gradient descent	$\theta \leftarrow \theta - \eta \text{ grad}$	<code>optimizer.step()</code>
Gradient reset	Clear old gradients	<code>optimizer.zero_grad()</code>
Parameter storage	W, b	<code>model.parameters()</code>
Forward propagation	Layer-by-layer computation	<code>model(X)</code>

The most important connection: PyTorch does not change the underlying mathematics. It gives you abstractions that make the mathematics scalable and practical.

21. Common Mistakes and Debugging Checklist

21.1 Shape mismatch

```
print(X.shape)
print(y.shape)
print(model(X).shape)
```

If a matrix multiplication or layer fails, inspect the shapes first.

21.2 Forgetting zero_grad

If you repeatedly call `backward()` without clearing gradients, gradients accumulate. The standard training pattern is `optimizer.zero_grad(), loss.backward(), optimizer.step()`.

21.3 Applying sigmoid before BCEWithLogitsLoss

Do not normally do both. If using `BCEWithLogitsLoss`, return logits from the model and apply sigmoid only when you need probabilities for interpretation.

21.4 Forgetting eval mode

Use `model.eval()` before inference, especially when your model contains Dropout or BatchNorm.

21.5 Forgetting no_grad

Use `torch.no_grad()` during inference when gradients are not needed.

21.6 CPU/GPU mismatch

Make sure model and tensors are on compatible devices.

```
print(next(model.parameters()).device)
print(X.device)
```

21.7 Wrong label dtype for CrossEntropyLoss

`CrossEntropyLoss` normally expects integer class indices. Do not automatically convert those labels to `float32`.

21.8 Confusing element-wise and matrix multiplication

```
a * b    # element-wise multiplication
A @ B    # matrix multiplication
```

22. Day 6 Exercises and Completion Checklist

Do not treat this guide as something to read once. The goal is to reproduce the ideas in code.

Exercise 1 - Tensor fundamentals

- Create tensors with `zeros`, `ones`, `rand`, and `randn`.
- Inspect `shape`, `ndim`, and `dtype`.
- Practice indexing and slicing.
- Reshape a 2 x 6 tensor into a 3 x 4 tensor.

Exercise 2 - Matrix multiplication

- Create X with shape [100, 5].

- Create W with shape [5, 3].
- Compute $X @ W$.
- Explain why the output has shape [100, 3].

Exercise 3 - Autograd

- Create x with requires_grad=True.
- Define $y = x^3 + 2x$.
- Call `y.backward()`.
- Check `x.grad` and verify it manually.

Exercise 4 - Inspect a model

- Create a model with `Linear(4, 8), ReLU, Linear(8, 1)`.
- Print the model.
- Print named parameters and their shapes.
- Count the total number of parameters.

Exercise 5 - Training loop

- Train a binary classifier with `BCEWithLogitsLoss`.
- Use SGD first.
- Print the loss every fixed number of epochs.
- Inspect at least one parameter's gradient.

Exercise 6 - Mini-batch training

- Wrap your tensors in `TensorDataset`.
- Create a `DataLoader` with a small batch size.
- Train using the `DataLoader` rather than the entire dataset.

Exercise 7 - Evaluation

- Switch to `model.eval()`.
- Use `torch.no_grad()`.
- Generate logits, probabilities, and thresholded predictions.
- Calculate accuracy and inspect a confusion matrix.

Exercise 8 - Persistence

- Save `model.state_dict()` to a .pth file.
- Create a fresh model instance.
- Load the saved `state_dict`.
- Verify that the loaded model produces the same predictions.

Day 6 completion checklist

- ☐ I can explain what a tensor is.
- ☐ I can read and reason about tensor shapes.
- ☐ I understand element-wise vs matrix multiplication.
- ☐ I understand `requires_grad`.
- ☐ I can explain what `loss.backward()` does.
- ☐ I understand where gradients are stored.
- ☐ I can define an `nn.Module`.
- ☐ I understand `nn.Linear`.
- ☐ I can choose a basic loss for classification or regression.
- ☐ I understand `optimizer.zero_grad()`.
- ☐ I understand `optimizer.step()`.
- ☐ I can write a complete training loop.
- ☐ I understand `Dataset` and `DataLoader`.
- ☐ I understand train / validation / test roles.
- ☐ I know when to use `model.train()` and `model.eval()`.
- ☐ I know why `torch.no_grad()` is used.
- ☐ I can move tensors and models to a device.
- ☐ I can evaluate a classifier.
- ☐ I can save and reload a model.
- ☐ I can translate PyTorch operations back into the mathematics from Days 2-5.

Final Mental Model

If you remember only one framework for PyTorch, remember this:

```
DATA
|
v
Tensor / DataLoader
|
v
model(x)
|
v
Predictions / logits
|
v
Loss function
|
v
loss.backward()
|
v
Gradients
|
v
```

```
optimizer.step()  
|  
v  
Updated parameters  
|  
+----- repeat -----+
```

And translate each stage back to the mathematics: the model performs parameterized functions, the loss measures error, backpropagation applies the chain rule, and the optimizer updates parameters to reduce future error.

Day 6 goal: You are not expected to know every PyTorch API. You should understand the core execution model well enough to read, write, debug, and explain a basic PyTorch training pipeline.